École Nationale des Sciences Appliquées de Beni Mellal
Université Sultan Moulay Slimane

Filière : Intelligence Artificielle et Cybersécurité

Module : BLOCKCHAIN

Projet : plateforme de financement participatif décentralisée à gouvernance par jeton

1. Contexte et objectifs pédagogiques

Ce projet mobilise l'ensemble des connaissances du module Blockhain autour de la réalisation d'une application décentralisée (dApp) complète, du contrat intelligent jusqu'à l'interface utilisateur, déployée sur un réseau de test compatible EVM.

Le projet impose une architecture multi-contrats avec gouvernance et gestion de fonds, ainsi que des exigences de sécurité strictes construites autour des vulnérabilités réelles étudiées en cours (reentrance, contrôle d'accès, manipulation de gouvernance). L'étudiant doit démontrer une maîtrise à la fois conceptuelle et pratique.

Objectifs pédagogiques visés

- Concevoir et implémenter un système **multi-contrats** en Solidity respectant un standard ERC.
- Mettre en œuvre les mécanismes de sécurité : contrôle d'accès, protection anti-reentrance, circuit breaker, multisignature.
- Développer une **interface dApp** interagissant avec les contrats via une bibliothèque Web3 et un wallet (MetaMask).
- Tester, auditer et documenter rigoureusement son propre code.

2. Sujet du projet

Une organisation souhaite lancer une plateforme décentralisée de financement participatif, sans intermédiaire centralisé. Les porteurs de projet créent des campagnes ; les contributeurs financent en ETH et reçoivent en retour un jeton de gouvernance ERC-20 (« CFD ») proportionnel à leur apport. Les décisions sensibles (déblocage anticipé de fonds, gel d'une campagne suspecte) sont tranchées par un vote pondéré par la détention de jetons CFD, plutôt que par un administrateur unique.

3. Spécifications fonctionnelles obligatoires

Module	Exigences fonctionnelles minimales
M1 — Jeton de gouvernance (ERC-20)	Jeton « CFD » via OpenZeppelin ; frappe (mint) automatique lors d'une contribution, selon un taux à définir et justifier.
M2 — Cœur du financement (Crowdfunding Core)	createCampaign(objectif, durée, bénéficiaire) ; contribute(id) ; statuts (En cours / Réussie / Échouée / Annulée) ; refund() si objectif non atteint ; withdrawFunds() réservé au bénéficiaire, conditionné à la validation par gouvernance.
M3 — Gouvernance (DAO)	Vote pondéré par solde de CFD pour valider un déblocage anticipé ou geler une campagne litigieuse ; quorum et majorité à définir et justifier ; délai de vote avant exécution automatique.
M4 — Interface dApp	Front-end (React/Vue/HTML-JS libre) connecté à MetaMask via Ethers.js ou Web3.js ; liste des campagnes, formulaire de contribution, tableau de bord de vote.

4. Exigences techniques imposées

Élément	Choix imposé / cadre
Langage & librairies	Solidity ^0.8.x + OpenZeppelin (ERC20, AccessControl, ReentrancyGuard)
Environnement de dev	Hardhat ou Foundry (au choix, à justifier) ; Remix toléré pour le prototypage
Réseau de déploiement	Testnet Sepolia (ou blockchain locale Hardhat pour la démonstration)
Interaction front-end	Ethers.js (ou Web3.js) + MetaMask, via l'ABI générée à la compilation
Métadonnées de campagne	Stockage sur IPFS (ex. Pinata) ; seul le hash IPFS est stocké on-chain
Tests	Hardhat/Foundry : tests unitaires + au moins un scénario de fuzzing

5. Exigences de sécurité obligatoires

Le contrat sera évalué comme s'il devait gérer de vrais fonds. Chaque point ci-dessous doit être implémenté et justifié dans le rapport par référence explicite à une notion de sécurité.

- **Anti-reentrance :** pattern Checks-Effects-Interactions et/ou **ReentrancyGuard** sur toutes les fonctions de retrait ; fournir un test qui simule une attaque par reentrance et prouve son échec.
- **Contrôle d'accès :** Ownable/AccessControl strict sur toutes les fonctions d'administration.
- **Circuit breaker :** pause d'urgence (pausable) activable par un rôle Admin, suspendant contributions et retraits en cas d'incident.
- **Multisignature (2-sur-3 minimum) :** obligatoire pour tout retrait de trésorerie au-delà d'un seuil défini par l'équipe.
- **Anti-manipulation de la gouvernance :** le pouvoir de vote doit être calculé sur un solde figé (snapshot) au moment de la création du vote — et non au moment du vote — afin de contrer une prise de contrôle éclair par acquisition temporaire de jetons.
- **Audit statique :** au moins un outil (Slither ou Mythril) exécuté sur les contrats finaux ; le rapport de findings et les corrections apportées sont annexés au livrable.

6. Fonctionnalité bonus (points supplémentaires)

Intégrer un script d'analyse hors-chaîne (Python) appliquant une heuristique simple de détection d'anomalies (montants répétitifs, adresses liées, pics de contribution) sur l'historique des transactions du contrat — en lien avec IA au service de la blockchain / détection de fraude).

7. Livrables attendus

- **Dépôt Git** structuré : /contracts, /test, /frontend, /scripts, /docs.
- **Rapport écrit** (10 à 15 pages) : cahier des charges, architecture, choix techniques, analyse de sécurité, résultats des tests et de l'audit, captures d'écran du déploiement testnet.
- **Soutenance orale** (10 min) avec questions ciblées sur la sécurité et les choix d'architecture.

8. Modalités pratiques

- Travail en Binome/trinôme.
- Durée indicative : 7 semaines.
- Le générateur de contrats OpenZeppelin Wizard est toléré comme point de départ, mais la **logique métier doit être originale** et pleinement comprise par l'équipe.
- Tout code copié sans compréhension démontrable sera sanctionné lors de la soutenance.